Putting it together: from messy data to a summary table

Wednesday, June 3, 2026

Today: Wednesday June 3

Before class

- **No new reading**

Plan for today

- No new verbs. Today is about **combining and sequencing** them
- The workflow: question –> target table –> plan –> build –> **check**
- Working on a genuinely messy dataset (`flights`)
- Handling missing values, small groups, wrong order

Helpful links

- Syllabus · AI policy · Schedule · *R4DS* Ch. 3 · *DC* Ch. 7 · *DC* Ch. 10 · dplyr reference

What you already have

Single-table toolkit:

Table 1: Six verbs, plus `count()` and the pipe.

Touches	Verbs
Rows	`filter()`, `arrange()`
Columns	`select()`, `mutate()`
Groups	`group_by()`, `summarise()`

> **Tip**
>
> Today we will practice *choosing which verb, in what order*, and *checking that the answer is trustworthy and rigorous.*

Workflow

1. **State the question** in plain English.
2. **Sketch the target table:** what does one row represent? what columns?
3. **Plan the verbs** as English sentences, writing one sentence per step.
4. **Build the pipeline** one verb at a time, looking at the output each time.
5. **Check it.** Does the count make sense? Are missing values handled? Does the result match your intuition/pass a sanity check?

> **Note**
>
> Make sure not to skip Step 5. A pipeline that *runs* is not the same as a pipeline that's *correct*.

A messier dataset

`flights`

You saw `flights` on HW2 and Friday; it's also the running example in R4DS Ch. 3. It contains data on every flight out of NYC area airports in 2013:

```
glimpse(flights)
```

```
Rows: 336,776
Columns: 19
$ year           <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2~
$ month          <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ day            <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1~
$ dep_time       <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558, 558, ~
$ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600, 600, ~
$ dep_delay      <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2, -2, -
1~
$ arr_time       <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753, 849,~
$ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745, 851,~
```

```
$ arr_delay      <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3, 7, -
1~
$ carrier        <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV", "B6", "~
$ flight         <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79, 301, 4~
$ tailnum        <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN", "N394~
$ origin         <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR", "LGA",~
$ dest           <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL", "IAD",~
$ air_time       <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138, 149, 1~
$ distance       <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944, 733, ~
$ hour           <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6, 6, 6~
$ minute         <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 0, 59, 0~
$ time_hour      <dttm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-
01 0~
```

Dirty data

This is a common example of what a real dataset often looks like immediately after you load it into R, before you've done any data cleaning:

- **336,776 rows, 19 columns**: `select()` and `n()` are much more relevant here compared to a smaller dataset like `penguins`.
- **Meaningful missing values:** cancelled flights have `NA` for `dep_delay` and `arr_delay`.
- **Codes, rather than long-form labels:** here, `carrier` is `"UA"`, not `"United"`. (Turning codes into names requires a *join*, which is later, so today we'll stick to the codes.)

```
flights |>
  summarise(
    n           = n(),
    missing_dep = sum(is.na(dep_delay))
  )
```

```
# A tibble: 1 x 2
       n missing_dep
   <int>       <int>
1 336776        8255
```

> **Warning**
>
> The above result tells us that there are 8,000 rows with a missing departure delay. Every average we compute has to decide what to do with them, or it silently returns `NA`.

Worked example 1

Q: which months had the worst delays?

Plan:

- We want one row per **month**, with its average departure delay and a count.
- *Group by* month, *then summarise* the mean and `n()`, *then arrange* worst-first.

Target table: `month | avg_delay | n`, which should have twelve rows.

```
flights |>
  group_by(month) |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  ) |>
  arrange(desc(avg_delay))
```

```
# A tibble: 12 x 3
   month avg_delay     n
   <int>     <dbl> <int>
 1     7     21.7  29425
 2     6     20.8  28243
 3    12     16.6  28135
 4     4     13.9  28330
 5     3     13.2  28834
 6     5     13.0  28796
 7     8     12.6  29327
 8     2     10.8  24951
 9     1     10.0  27004
10     9      6.72 27574
11    10      6.24 28889
12    11      5.44 27268
```

What just happened

We used **three** verbs in sequence, and two safety habits from this week:

- `na.rm = TRUE` so the cancelled flights don't give `NA`s for the means.

- `n = n()` so we can see each average rests on around 25,000 flights at least, rather than a small number.

> **Tip**
>
> Summer months (6, 7) and December rise to the top, which matches what you'd expect from thunderstorms and holiday travel. This is a good place to think about common sense/sanity checking; if January had come out worst by a mile (no thunderstorms and little holiday travel), it might suggest something is wrong with your code. At the very least, it's a signal to double check your code/investigate further.

Worked example 2

Q: worst JFK summer destinations?

Question: for flights leaving **JFK in summer**, which **destination** had the worst average arrival delay, among destinations with **enough flights to trust the result**? Let's set an arbitrary cutoff at 100 flights for simplicity.

Plan:

- *Filter* to JFK, summer months.
- *Group by* destination, *summarise* mean arrival delay + `n()`.
- *Filter again* to drop tiny destinations.
- *Arrange* worst-first.

Execution

```
flights |>
  filter(origin == "JFK", month %in% c(6, 7, 8)) |>
  group_by(dest) |>
  summarise(
    avg_arr_delay = mean(arr_delay, na.rm = TRUE),
    n             = n(),
    .groups       = "drop" # <-- removes grouping metadata from your data frame
  ) |>
  filter(n >= 100) |>
  arrange(desc(avg_arr_delay))
```

```
# A tibble: 49 x 3
   dest  avg_arr_delay     n
   <chr>         <dbl> <int>
 1 CMH            44.4   194
 2 CVG            39.4   267
 3 IND            30.8   144
 4 PDX            29.6   265
 5 BNA            28.8   184
 6 DEN            28.6   184
 7 CLE            28.2   199
 8 ORF            26.7   128
 9 PBI            25.9   368
10 JAX            22.1   365
# i 39 more rows
```

> **Note**
>
> Notice that `filter()` appears **twice**, operating once on the raw rows (which flights), and once on the summary table (which destinations are "big enough").

The necessity of the second `filter ()`.scrollable .nostretch}

Without `filter(n >= 100)`, the "worst destination" could be whichever one had a single very-late flight (isn't necessarily a problem with this dataset, but we got lucky). **A mean from a group of 3 is not very reliable.**

- If we gained more samples, the answer is much more likely to shift significantly in the case of a small sample size.

It's a good idea to have verification steps built-in to all of your pipelines when possible; always including the count column is how you *catch* the problem before it ends up in a report that you have to retract later.

Worked example 3

Q: do long hauls leave later?

Now we can leverage `mutate()` before piping our transformed data into a plot.

English plan: *mutate* a long/short label from `distance`, *group by* it, *summarise* the average delay, *then plot* the summary table.

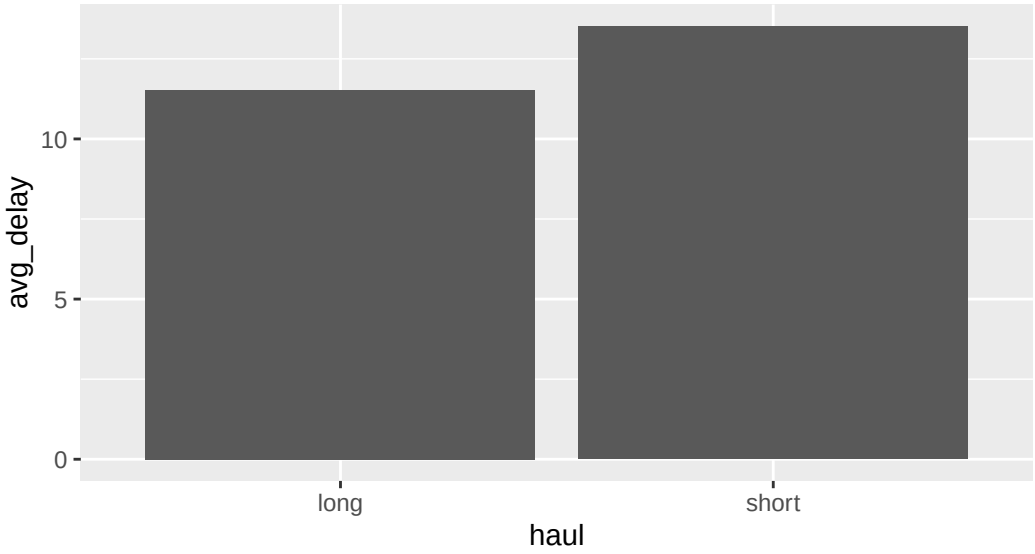
```r
flights |>
  mutate(haul = ifelse(distance > 1000, "long", "short")) |>
  group_by(haul) |>
  summarise(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n         = n(),
    .groups   = "drop"
  )
```

```
# A tibble: 2 x 3
  haul  avg_delay      n
  <chr>     <dbl>  <int>
1 long       11.5 147105
2 short      13.5 189671
```

From summary table to plot

Because the summary is just a (tiny) data frame, it's already glyph-ready, so we can pipe it into `ggplot()`:

```r
flights |>
  mutate(haul = ifelse(distance > 1000, "long", "short")) |>
  group_by(haul) |>
  summarise(avg_delay = mean(dep_delay, na.rm = TRUE), .groups = "drop") |>
  ggplot(aes(x = haul, y = avg_delay)) +
  geom_col()
```



Getting it right

Order matters

The same verbs in a different order answer different questions:

Filter, *then* summarise

"Average delay *of JFK flights.*" (This is more computationally efficient)

```
flights |>
  filter(origin == "JFK") |>
  summarise(
    avg = mean(dep_delay,
               na.rm = TRUE)
  )
```

Summarise, *then* filter

"Of the per-origin averages, keep JFK's."

```
flights |>
  group_by(origin) |>
  summarise(
    avg = mean(dep_delay,
```

```
                na.rm = TRUE)
  ) |>
  filter(origin == "JFK")
```

Both are valid, and both give the same number in this case. However, they won't always be. The trick is knowing **which question you're actually asking** before you write it.

A case when the output changes with order

```
flights |>
  filter(dep_delay > 60) |>
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE))
```

```
# A tibble: 1 x 1
  avg_delay
      <dbl>
1      119.
```

```
flights |>
  summarise(avg_delay = mean(arr_delay, na.rm = TRUE)) |>
  filter(avg_delay > 60)
```

```
# A tibble: 0 x 1
# i 1 variable: avg_delay <dbl>
```

The three checks

Before trusting any wrangled result, run this checklist:

Table 2: Your verification checklist.

Check	The verb/habit
Did missing values get handled?	`na.rm`, `!is.na()`
Is each number based on enough rows?	`n = n()`
Does the answer pass a smell test?	compare to what you know

> **Warning**
>
> A pipeline that runs without error has cleared the *lowest* bar; that doesn't *necessarily* mean everything is fine.

What we did today

- **No new verbs**: just the six from Monday and Tuesday, sequenced
- **The workflow:** question –> target table –> English plan –> build incrementally –> check
- **Real mess:** missing delays, code-not-name columns, tiny groups
- **`filter()` in two roles:** subsetting raw rows *and* trimming a summary table
- **Order matters:** filter-then-summarise =/= summarise-then-filter
- **The full arc:** raw rows –> glyph-ready summary –> plot

Activity

Activity: a flights investigation

Roughly 25 minutes. Individually or with a neighbor.

- Activity Canvas Link

Format: you'll answer a series of questions about `flights` by building pipelines, but each one starts with writing the **plan in words first**. You'll compare two pipelines that look alike but answer different questions, catch a silently-dropped `NA`, and finish with one open-ended question where you choose the verbs and **report your sanity checks**.

To turn in: save your `.qmd` and rendered HTML.

Wrap-up & looking ahead

Tomorrow (Thursday): new reading and new content, reshaping data between **wide and narrow** with `pivot_longer()` and `pivot_wider()`.

Upcoming Deadlines

- **Thu 6/4, before class**: *DC* §12.1
- **Thu 6/4, 11:59 p.m.**: Project: Team formation
- **Fri 6/5, 11:59 p.m.**: Homework 3: Data Wrangling & Reshaping

Reach out

Stuck? Office hours, or email me (cgc5478@psu.edu) or Xinyue (xpw5228@psu.edu).

Reference

Syllabus · AI policy · Schedule · *R4DS* Ch. 3 · *DC* Ch. 10 · dplyr reference